

CSA0624 - Design and Analysis of Algorithms

Name: Aryan Arun

Reg.No: 192411157

Searching Algorithms Lab Record

Sequential Search & Brute Force String Matching

SEQUENTIAL SEARCH

Question 1: Basic Sequential Search

Input:

- Array = 12, 25, 8, 45, 32, 19, 50
- Key = 32

Python Code

```
def sequential_search(arr, key):
    comparisons = 0
    for i in range(len(arr)):
        comparisons += 1
        if arr[i] == key:
            return i + 1, comparisons    # 1-indexed position
    return -1, comparisons

arr = [12, 25, 8, 45, 32, 19, 50]
key = 32

pos, comparisons = sequential_search(arr, key)
print("Array:", arr)
print("Key:", key)
if pos != -1:
    print("Element found at position", pos)
else:
    print("Element not found")
print("Number of comparisons =", comparisons)
```

Output

```
Array: [12, 25, 8, 45, 32, 19, 50]
Key: 32
Element found at position 5
Number of comparisons = 5
```

Question 2: Unsuccessful Search

Input:

- Array = 5, 10, 15, 20, 25, 30, 35
- Key = 18

Python Code

```
def sequential_search(arr, key):
    comparisons = 0
    for i in range(len(arr)):
        comparisons += 1
        if arr[i] == key:
            return i + 1, comparisons
    return -1, comparisons

arr = [5, 10, 15, 20, 25, 30, 35]
key = 18

pos, comparisons = sequential_search(arr, key)
print("Array:", arr)
print("Key:", key)
if pos != -1:
    print("Element found at position", pos)
else:
    print("Element not found")
print("Number of comparisons =", comparisons)
```

Output

```
Array: [5, 10, 15, 20, 25, 30, 35]
Key: 18
Element not found
Number of comparisons = 7
```

Question 3: First Occurrence Search

Input:

- Array = 10, 25, 15, 25, 30, 25, 40
- Key = 25

Python Code

```
def first_occurrence_search(arr, key):
    comparisons = 0
    for i in range(len(arr)):
        comparisons += 1
        if arr[i] == key:
            return i + 1, comparisons
    return -1, comparisons

arr = [10, 25, 15, 25, 30, 25, 40]
key = 25

pos, comparisons = first_occurrence_search(arr, key)
print("Array:", arr)
print("Key:", key)
if pos != -1:
    print("First occurrence at position", pos)
else:
    print("Element not found")
print("Number of comparisons =", comparisons)
```

Output

```
Array: [10, 25, 15, 25, 30, 25, 40]
Key: 25
First occurrence at position 2
Number of comparisons = 2
```

Question 4: Find All Occurrences

Input:

- Array = 7, 12, 7, 25, 18, 7, 30, 7
- Key = 7

Python Code

```
def all_occurrences_search(arr, key):
    comparisons = 0
    positions = []
    for i in range(len(arr)):
        comparisons += 1
        if arr[i] == key:
            positions.append(i + 1)
    return positions, comparisons

arr = [7, 12, 7, 25, 18, 7, 30, 7]
key = 7

positions, comparisons = all_occurrences_search(arr, key)
print("Array:", arr)
print("Key:", key)
print("Occurrences at positions:")
print(", ".join(map(str, positions)))
print("Total occurrences =", len(positions))
print("Number of comparisons =", comparisons)
```

Output

```
Array: [7, 12, 7, 25, 18, 7, 30, 7]
Key: 7
Occurrences at positions:
1, 3, 6, 8
Total occurrences = 4
Number of comparisons = 8
```

Question 5: Count Comparisons, Matches and Mismatches

Input:

- Array = 3, 6, 9, 12, 15, 18, 21
- Key = 15

Python Code

```
def sequential_search_stats(arr, key):
    comparisons = 0
    matches = 0
    mismatches = 0
    found_pos = -1
    for i in range(len(arr)):
        comparisons += 1
        if arr[i] == key:
            matches += 1
            if found_pos == -1:
                found_pos = i + 1
        else:
            mismatches += 1
    return found_pos, comparisons, matches, mismatches

arr = [3, 6, 9, 12, 15, 18, 21]
key = 15

pos, comparisons, matches, mismatches = sequential_search_stats(arr, key)
print("Array:", arr)
print("Key:", key)
print("Position found:", pos if pos != -1 else "Not found")
print("Total comparisons =", comparisons)
print("Total matches =", matches)
print("Total mismatches =", mismatches)
```

Output

```
Array: [3, 6, 9, 12, 15, 18, 21]
Key: 15
Position found: 5
Total comparisons = 7
Total matches = 1
Total mismatches = 6
```

Question 6: Sentinel Sequential Search vs Ordinary Sequential Search

Input:

- Array = 14, 9, 22, 35, 18, 41, 27
- Key = 18

Python Code

```
def ordinary_sequential_search(arr, key):
    comparisons = 0
    for i in range(len(arr)):
        comparisons += 1
        if arr[i] == key:
            return i + 1, comparisons
    return -1, comparisons

def sentinel_sequential_search(arr, key):
    n = len(arr)
    temp = arr[:]          # copy so original array is untouched
    last = temp[n - 1]
    temp[n - 1] = key      # place sentinel at the end
    comparisons = 0
    i = 0
    while temp[i] != key:
        comparisons += 1
        i += 1
    comparisons += 1       # final comparison that matched (either sentinel or real element)
    if i < n - 1 or last == key:
        return i + 1, comparisons
    return -1, comparisons

arr = [14, 9, 22, 35, 18, 41, 27]
key = 18

pos_o, comp_o = ordinary_sequential_search(arr, key)
pos_s, comp_s = sentinel_sequential_search(arr, key)

print("Array:", arr)
print("Key:", key)
print()
print("--- Ordinary Sequential Search ---")
print("Position found:", pos_o)
print("Comparisons:", comp_o)
print()
print("--- Sentinel Sequential Search ---")
print("Position found:", pos_s)
print("Comparisons:", comp_s)
print()
print("--- Comparison ---")
print(f"Ordinary search comparisons: {comp_o}")
print(f"Sentinel search comparisons: {comp_s}")
print("Sentinel search avoids the explicit 'i < n' boundary check on every")
print("iteration by guaranteeing a match at the sentinel position, trading a")
print("small amount of setup work for fewer per-iteration comparisons in a")
print("low-level (e.g. array-bounds-checked) implementation.")
```

Output

```
Array: [14, 9, 22, 35, 18, 41, 27]
Key: 18

--- Ordinary Sequential Search ---
Position found: 5
Comparisons: 5

--- Sentinel Sequential Search ---
Position found: 5
Comparisons: 5

--- Comparison ---
Ordinary search comparisons: 5
Sentinel search comparisons: 5
Sentinel search avoids the explicit 'i < n' boundary check on every
iteration by guaranteeing a match at the sentinel position, trading a
small amount of setup work for fewer per-iteration comparisons in a
low-level (e.g. array-bounds-checked) implementation.
```

Notes / Analysis

Sentinel search places the key at the end of the array (the sentinel), which removes the need to separately check the array-bounds condition ($i < n$) on every iteration. For this input both approaches perform the same number of comparisons because the key was found well before the end of the array; the saving from the sentinel technique becomes more apparent in low-level implementations where the bounds check is itself a comparison.

Question 7: Sequential Search on Register Numbers

Input:

- Register Numbers = 101, 102, 103, 104, 105, 106
- Search Key = 104

Python Code

```
def sequential_search(arr, key):
    comparisons = 0
    for i in range(len(arr)):
        comparisons += 1
        if arr[i] == key:
            return i + 1, comparisons
    return -1, comparisons

register_numbers = [101, 102, 103, 104, 105, 106]
search_key = 104

pos, comparisons = sequential_search(register_numbers, search_key)
print("Register Numbers:", register_numbers)
print("Search Key:", search_key)
if pos != -1:
    print("Register Number found at position", pos)
else:
    print("Register Number not found")
print("Number of comparisons =", comparisons)
```

Output

```
Register Numbers: [101, 102, 103, 104, 105, 106]
Search Key: 104
Register Number found at position 4
Number of comparisons = 4
```

Question 8: Sequential Search on String Data

Input:

- Names = Anu, Bala, Charan, Deepa, Esha, Farhan
- Search Name = Deepa

Python Code

```
def sequential_search_string(arr, key):
    comparisons = 0
    for i in range(len(arr)):
        comparisons += 1
        if arr[i] == key:
            return i + 1, comparisons
    return -1, comparisons

names = ["Anu", "Bala", "Charan", "Deepa", "Esha", "Farhan"]
search_name = "Deepa"

pos, comparisons = sequential_search_string(names, search_name)
print("Names:", names)
print("Search Name:", search_name)
if pos != -1:
    print("Name found at position", pos)
else:
    print("Name not found")
print("Number of comparisons =", comparisons)
```

Output

```
Names: ['Anu', 'Bala', 'Charan', 'Deepa', 'Esha', 'Farhan']
Search Name: Deepa
Name found at position 4
Number of comparisons = 4
```

Question 9: Sequential Search on a 2-D Matrix

Input:

- Matrix = [[12, 8, 15], [5, 18, 27], [9, 11, 24]]
- Key = 24

Python Code

```
def search_2d_matrix(matrix, key):
    comparisons = 0
    for r in range(len(matrix)):
        for c in range(len(matrix[r])):
            comparisons += 1
            if matrix[r][c] == key:
                return (r + 1, c + 1), comparisons    # 1-indexed row, column
    return None, comparisons

matrix = [
    [12, 8, 15],
    [5, 18, 27],
    [9, 11, 24]
]
key = 24

pos, comparisons = search_2d_matrix(matrix, key)
print("Matrix:")
for row in matrix:
    print(row)
print("Key =", key)
if pos:
    print(f"Element found at Row {pos[0]} Column {pos[1]}")
else:
    print("Element not found")
print("Number of comparisons =", comparisons)
```

Output

```
Matrix:
[12, 8, 15]
[5, 18, 27]
[9, 11, 24]
Key = 24
Element found at Row 3 Column 3
Number of comparisons = 9
```

Question 10: Full Performance Analysis of Sequential Search

Input:

- Array = 45, 23, 67, 12, 89, 34, 56, 78, 90, 11, 29, 73, 18, 64, 37
- Search Keys = 73, 18, 100

Python Code

```
def sequential_search_verbose(arr, key):
    comparisons = 0
    log = []
    for i in range(len(arr)):
        comparisons += 1
        matched = (arr[i] == key)
        log.append((i + 1, arr[i], matched))
        if matched:
            return i + 1, comparisons, log
    return -1, comparisons, log

arr = [45, 23, 67, 12, 89, 34, 56, 78, 90, 11, 29, 73, 18, 64, 37]
keys = [73, 18, 100]

print("Array:", arr)
print("Array size (n):", len(arr))
print()

results = {}
for key in keys:
    pos, comparisons, log = sequential_search_verbose(arr, key)
    results[key] = (pos, comparisons)
    print(f"--- Searching for key = {key} ---")
    for step, (position, value, matched) in enumerate(log, start=1):
        result = "MATCH" if matched else "mismatch"
        print(f"Comparison {step}: arr[{position}] = {value} vs key = {key} -> {result}")
    if pos != -1:
        print(f"Result: Found at position {pos}")
    else:
        print("Result: Not found")
    print(f"Total comparisons: {comparisons}")
    print()

print("---- Summary ----")
for key in keys:
    pos, comparisons = results[key]
    status = f"found at position {pos}" if pos != -1 else "not found"
    print(f"Key {key:>4}: {status}, comparisons = {comparisons}")

n = len(arr)
print()
print("---- Case Identification ----")
print(f"Key 73 -> comparisons = {results[73][1]} (element near middle/later part of array)")
print(f"Key 18 -> comparisons = {results[18][1]} (element found later in array)")
print(f"Key 100 -> comparisons = {results[100][1]} (element absent, scans entire array -> WORST CASE, n = {n})")
print()
print("Best case : key present at position 1 -> 1 comparison -> O(1)")
print("Worst case : key absent OR present at the last position -> n comparisons -> O(n)")
```

```

print("Average case: key present at a random position      -> ~(n+1)/2 comparisons ->
0(n)")

print()
print("---- Complexity Analysis ----")
print("Best-case time complexity   : O(1)")
print("Average-case time complexity: O(n)")
print("Worst-case time complexity   : O(n)")
print("Space complexity             : O(1) additional space (no extra data structures
used)")

```

Output

```

Array: [45, 23, 67, 12, 89, 34, 56, 78, 90, 11, 29, 73, 18, 64, 37]
Array size (n): 15

```

```

--- Searching for key = 73 ---
Comparison 1: arr[1] = 45 vs key = 73 -> mismatch
Comparison 2: arr[2] = 23 vs key = 73 -> mismatch
Comparison 3: arr[3] = 67 vs key = 73 -> mismatch
Comparison 4: arr[4] = 12 vs key = 73 -> mismatch
Comparison 5: arr[5] = 89 vs key = 73 -> mismatch
Comparison 6: arr[6] = 34 vs key = 73 -> mismatch
Comparison 7: arr[7] = 56 vs key = 73 -> mismatch
Comparison 8: arr[8] = 78 vs key = 73 -> mismatch
Comparison 9: arr[9] = 90 vs key = 73 -> mismatch
Comparison 10: arr[10] = 11 vs key = 73 -> mismatch
Comparison 11: arr[11] = 29 vs key = 73 -> mismatch
Comparison 12: arr[12] = 73 vs key = 73 -> MATCH
Result: Found at position 12
Total comparisons: 12

```

```

--- Searching for key = 18 ---
Comparison 1: arr[1] = 45 vs key = 18 -> mismatch
Comparison 2: arr[2] = 23 vs key = 18 -> mismatch
Comparison 3: arr[3] = 67 vs key = 18 -> mismatch
Comparison 4: arr[4] = 12 vs key = 18 -> mismatch
Comparison 5: arr[5] = 89 vs key = 18 -> mismatch
Comparison 6: arr[6] = 34 vs key = 18 -> mismatch
Comparison 7: arr[7] = 56 vs key = 18 -> mismatch
Comparison 8: arr[8] = 78 vs key = 18 -> mismatch
Comparison 9: arr[9] = 90 vs key = 18 -> mismatch
Comparison 10: arr[10] = 11 vs key = 18 -> mismatch
Comparison 11: arr[11] = 29 vs key = 18 -> mismatch
Comparison 12: arr[12] = 73 vs key = 18 -> mismatch
Comparison 13: arr[13] = 18 vs key = 18 -> MATCH
Result: Found at position 13
Total comparisons: 13

```

```

--- Searching for key = 100 ---
Comparison 1: arr[1] = 45 vs key = 100 -> mismatch
Comparison 2: arr[2] = 23 vs key = 100 -> mismatch
Comparison 3: arr[3] = 67 vs key = 100 -> mismatch
Comparison 4: arr[4] = 12 vs key = 100 -> mismatch
Comparison 5: arr[5] = 89 vs key = 100 -> mismatch
Comparison 6: arr[6] = 34 vs key = 100 -> mismatch

```

```

Comparison 7: arr[7] = 56 vs key = 100 -> mismatch
Comparison 8: arr[8] = 78 vs key = 100 -> mismatch
Comparison 9: arr[9] = 90 vs key = 100 -> mismatch
Comparison 10: arr[10] = 11 vs key = 100 -> mismatch
Comparison 11: arr[11] = 29 vs key = 100 -> mismatch
Comparison 12: arr[12] = 73 vs key = 100 -> mismatch
Comparison 13: arr[13] = 18 vs key = 100 -> mismatch
Comparison 14: arr[14] = 64 vs key = 100 -> mismatch
Comparison 15: arr[15] = 37 vs key = 100 -> mismatch
Result: Not found
Total comparisons: 15

---- Summary ----
Key 73: found at position 12, comparisons = 12
Key 18: found at position 13, comparisons = 13
Key 100: not found, comparisons = 15

---- Case Identification ----
Key 73 -> comparisons = 12 (element near middle/later part of array)
Key 18 -> comparisons = 13 (element found later in array)
Key 100 -> comparisons = 15 (element absent, scans entire array -> WORST CASE, n = 15)

Best case : key present at position 1 -> 1 comparison -> O(1)
Worst case : key absent OR present at the last position -> n comparisons -> O(n)
Average case: key present at a random position ->  $\sim(n+1)/2$  comparisons -> O(n)

---- Complexity Analysis ----
Best-case time complexity : O(1)
Average-case time complexity: O(n)
Worst-case time complexity : O(n)
Space complexity : O(1) additional space (no extra data structures used)

```

Notes / Analysis

Best-case complexity: $O(1)$ — occurs when the search key is at the first position.

Average-case complexity: $O(n)$ — on average roughly $(n+1)/2$ comparisons are needed for a successful search.

Worst-case complexity: $O(n)$ — occurs when the key is at the last position (key = 18 in this run needed 13 comparisons) or is absent entirely (key = 100 needed all 15 comparisons, the true worst case here).

Space complexity: $O(1)$ — only a constant number of index/counter variables are used regardless of array size.

BRUTE FORCE STRING MATCHING

Question 1: Basic Brute Force String Matching

Input:

- Text = "AABAACAADAABAABA"
- Pattern = "AABA"

Python Code

```
def brute_force_search(text, pattern):
    n, m = len(text), len(pattern)
    comparisons = 0
    positions = []
    for i in range(n - m + 1):
        j = 0
        while j < m:
            comparisons += 1
            if text[i+j] != pattern[j]:
                break
            j += 1
        if j == m:
            positions.append(i)
    return positions, comparisons

text = "AABAACAADAABAABA"
pattern = "AABA"

positions, comparisons = brute_force_search(text, pattern)

print("Text:", text)
print("Pattern:", pattern)
print("Position(s) where pattern occurs:", positions)
print("Total number of comparisons:", comparisons)
```

Output

```
Text: AABAACAADAABAABA
Pattern: AABA
Position(s) where pattern occurs: [0, 9, 12]
Total number of comparisons: 30
```

Question 2: Find All Occurrences of a Pattern

Input:

- Text = "BANANABANANA"
- Pattern = "ANA"

Python Code

```
def brute_force_all_occurrences(text, pattern):
    n, m = len(text), len(pattern)
    comparisons = 0
    positions = []
    for i in range(n - m + 1):
        j = 0
        while j < m:
            comparisons += 1
            if text[i+j] != pattern[j]:
                break
            j += 1
        if j == m:
            positions.append(i)
    return positions, comparisons

text = "BANANABANANA"
pattern = "ANA"

positions, comparisons = brute_force_all_occurrences(text, pattern)

print("Text:", text)
print("Pattern:", pattern)
print("Occurrences at positions:", ", ".join(map(str, positions)))
print("Total comparisons:", comparisons)
```

Output

```
Text: BANANABANANA
Pattern: ANA
Occurrences at positions: 1, 3, 7, 9
Total comparisons: 19
```


Question 3: Comparison Table for Each Alignment

Input:

- Text = "MISSISSIPPI"
- Pattern = "ISSI"

Python Code

```
def brute_force_comparison_table(text, pattern):
    n, m = len(text), len(pattern)
    total_comparisons = 0
    table = []
    for i in range(n - m + 1):
        j = 0
        local_comp = 0
        while j < m:
            local_comp += 1
            total_comparisons += 1
            if text[i+j] != pattern[j]:
                break
            j += 1
        matched = (j == m)
        table.append((i, local_comp, matched))
    return table, total_comparisons

text = "MISSISSIPPI"
pattern = "ISSI"

table, total_comparisons = brute_force_comparison_table(text, pattern)

print("Text:", text)
print("Pattern:", pattern)
print()
print(f"{'Shift':<7}{'Comparisons':<14}{'Result'}")
for shift, comp, matched in table:
    result = "MATCH" if matched else "MISMATCH"
    print(f"{'shift':<7}{'comp':<14}{'result'}")

print()
print("Total comparisons across all shifts:", total_comparisons)
occurrences = [s for s, c, m in table if m]
print("Pattern occurs at position(s):", occurrences if occurrences else "None")
```

Output

```
Text: MISSISSIPPI
Pattern: ISSI
```

Shift	Comparisons	Result
0	1	MISMATCH
1	4	MATCH
2	1	MISMATCH
3	1	MISMATCH
4	4	MATCH
5	1	MISMATCH

6	1	MISMATCH
7	2	MISMATCH

Total comparisons across all shifts: 15

Pattern occurs at position(s): [1, 4]

Question 4: Count Total Comparisons, Matches and Mismatches

Input:

- Text = "ABABABABAB"
- Pattern = "ABAB"

Python Code

```
def brute_force_stats(text, pattern):
    n, m = len(text), len(pattern)
    total_comparisons = 0
    matches = 0
    mismatches = 0
    positions = []
    for i in range(n - m + 1):
        j = 0
        while j < m:
            total_comparisons += 1
            if text[i+j] != pattern[j]:
                mismatches += 1
                break
            else:
                matches += 1
            j += 1
        if j == m:
            positions.append(i)
    return positions, total_comparisons, matches, mismatches

text = "ABABABABAB"
pattern = "ABAB"

positions, total_comparisons, matches, mismatches = brute_force_stats(text, pattern)

print("Text:", text)
print("Pattern:", pattern)
print("Pattern occurs at position(s):", positions)
print()
print("Total character comparisons:", total_comparisons)
print("Total matches (characters that were equal):", matches)
print("Total mismatches (characters that differed):", mismatches)
```

Output

```
Text: ABABABABAB
Pattern: ABAB
Pattern occurs at position(s): [0, 2, 4, 6]

Total character comparisons: 19
Total matches (characters that were equal): 16
Total mismatches (characters that differed): 3
```

Question 5: Best / Average / Worst Case Analysis

Input:

- Text = "AAAAAAAAAB"
- Pattern = "AAAB"

Python Code

```
def brute_force_search(text, pattern):
    n, m = len(text), len(pattern)
    comparisons = 0
    positions = []
    for i in range(n - m + 1):
        j = 0
        while j < m:
            comparisons += 1
            if text[i+j] != pattern[j]:
                break
            j += 1
        if j == m:
            positions.append(i)
    return positions, comparisons

text = "AAAAAAAAAB"
pattern = "AAAB"
positions, comparisons = brute_force_search(text, pattern)

print("Text:", text)
print("Pattern:", pattern)
print("Occurrence position(s):", positions)
print("Total comparisons:", comparisons)

n, m = len(text), len(pattern)
best_case = m
worst_case = m * (n - m + 1)
print("\nBest case comparisons (if pattern matched immediately):", best_case)
print("Worst case comparisons (theoretical max, m*(n-m+1)):", worst_case)
print("Actual comparisons for this input:", comparisons)

if comparisons == best_case:
    case = "Best Case"
elif comparisons == worst_case:
    case = "Worst Case"
else:
    case = "Average Case"
print("This run represents:", case)
```

Output

```
Text: AAAAAAAAAAB
Pattern: AAAB
Occurrence position(s): [5]
Total comparisons: 30

Best case comparisons (if pattern matched immediately): 5
```

```
Worst case comparisons (theoretical max,  $m \cdot (n - m + 1)$ ): 30  
Actual comparisons for this input: 30  
This run represents: Worst Case
```

Notes / Analysis

Because the text consists mostly of the character 'A' repeated, almost every alignment matches several characters of the pattern before finally mismatching (or fully matching) — this forces the maximum possible number of comparisons at each shift. Hence, this input represents the WORST CASE for Brute Force String Matching, with comparisons equal to the theoretical maximum $m \times (n - m + 1)$.

Question 6: Report First Occurrence Only

Input:

- Text = "COMPUTERSCIENCE"
- Pattern = "SCI"

Python Code

```
def brute_force_first_occurrence(text, pattern):
    n, m = len(text), len(pattern)
    comparisons = 0
    for i in range(n - m + 1):
        j = 0
        while j < m:
            comparisons += 1
            if text[i+j] != pattern[j]:
                break
            j += 1
        if j == m:
            return i, comparisons
    return -1, comparisons

text = "COMPUTERSCIENCE"
pattern = "SCI"
pos, comparisons = brute_force_first_occurrence(text, pattern)

print("Text:", text)
print("Pattern:", pattern)
if pos != -1:
    print("First occurrence position:", pos)
else:
    print("Pattern not found")
print("Number of comparisons:", comparisons)
```

Output

```
Text: COMPUTERSCIENCE
Pattern: SCI
First occurrence position: 8
Number of comparisons: 11
```

Question 7: Case-Insensitive Search

Input:

- Text = "DataStructuresAndAlgorithms"
- Pattern = "ALGORITHMS"

Python Code

```
def brute_force_case_insensitive(text, pattern):
    text_l = text.lower()
    pattern_l = pattern.lower()
    n, m = len(text_l), len(pattern_l)
    comparisons = 0
    for i in range(n - m + 1):
        j = 0
        while j < m:
            comparisons += 1
            if text_l[i+j] != pattern_l[j]:
                break
            j += 1
        if j == m:
            return i, comparisons
    return -1, comparisons

text = "DataStructuresAndAlgorithms"
pattern = "ALGORITHMS"
pos, comparisons = brute_force_case_insensitive(text, pattern)

print("Text:", text)
print("Pattern:", pattern)
if pos != -1:
    print("Pattern found at position:", pos)
else:
    print("Pattern not found")
print("Number of comparisons:", comparisons)
```

Output

```
Text: DataStructuresAndAlgorithms
Pattern: ALGORITHMS
Pattern found at position: 17
Number of comparisons: 30
```

Question 8: Successful vs Unsuccessful Search

Input:

- Text = "PROGRAMMINGLAB"
- Pattern 1 = "LAB" (successful)
- Pattern 2 = "TEST" (unsuccessful)

Python Code

```
def brute_force_search(text, pattern):
    n, m = len(text), len(pattern)
    comparisons = 0
    positions = []
    for i in range(n - m + 1):
        j = 0
        while j < m:
            comparisons += 1
            if text[i+j] != pattern[j]:
                break
            j += 1
        if j == m:
            positions.append(i)
    return positions, comparisons

text = "PROGRAMMINGLAB"
pattern1 = "LAB"
pattern2 = "TEST"

pos1, comp1 = brute_force_search(text, pattern1)
pos2, comp2 = brute_force_search(text, pattern2)

print("Text:", text)
print("\n--- Successful Search ---")
print("Pattern:", pattern1)
print("Position(s) found:", pos1)
print("Comparisons:", comp1)

print("\n--- Unsuccessful Search ---")
print("Pattern:", pattern2)
print("Position(s) found:", pos2 if pos2 else "Not found")
print("Comparisons:", comp2)

print("\n--- Summary ---")
print(f"Successful search comparisons : {comp1}")
print(f"Unsuccessful search comparisons: {comp2}")
```

Output

```
Text: PROGRAMMINGLAB

--- Successful Search ---
Pattern: LAB
Position(s) found: [11]
Comparisons: 14
```



```
--- Unsuccessful Search ---  
Pattern: TEST  
Position(s) found: Not found  
Comparisons: 11  
  
--- Summary ---  
Successful search comparisons : 14  
Unsuccessful search comparisons: 11
```

Notes / Analysis

The successful search required more comparisons overall because it scanned through more of the text before locating the match near the end, and the final matching alignment itself needed 3 full character comparisons. The unsuccessful search terminates after scanning the full text once, with each shift mismatching quickly since 'TEST' shares very few characters in common with the text.

Question 9: Display Every Alignment

Input:

- Text = "ABCDABCABCD"
- Pattern = "ABCD"

Python Code

```
def brute_force_all_alignments(text, pattern):
    n, m = len(text), len(pattern)
    occurrences = []
    print(f"Text      : {text}")
    print(f"Pattern : {pattern}\n")
    for i in range(n - m + 1):
        j = 0
        match = True
        while j < m:
            if text[i+j] != pattern[j]:
                match = False
                break
            j += 1
        alignment_str = " " * i + pattern
        result = "MATCH" if match else "NO MATCH"
        print(f"Alignment {i+1:2d}: {alignment_str}  -> {result}")
        if match:
            occurrences.append(i)
    return occurrences

text = "ABCDABCABCD"
pattern = "ABCD"
occurrences = brute_force_all_alignments(text, pattern)

print("\nPattern occurrence position(s):", occurrences)
```

Output

```
Text      : ABCDABCABCD
Pattern : ABCD

Alignment  1: ABCD  -> MATCH
Alignment  2: ABCD  -> NO MATCH
Alignment  3:  ABCD  -> NO MATCH
Alignment  4:   ABCD  -> NO MATCH
Alignment  5:    ABCD  -> NO MATCH
Alignment  6:     ABCD  -> NO MATCH
Alignment  7:      ABCD  -> NO MATCH
Alignment  8:       ABCD  -> MATCH

Pattern occurrence position(s): [0, 7]
```

Question 10: Complete Program with Complexity Analysis

Input:

- Text = "TTATAGATCTCGTATTCTTTATAGATCTCCTATTCTT"
- Pattern = "TATCTT"

Python Code

```
def brute_force_full_analysis(text, pattern):
    n, m = len(text), len(pattern)
    comparisons = 0
    occurrences = []
    shift_log = []

    for i in range(n - m + 1):
        j = 0
        local_comp = 0
        while j < m:
            local_comp += 1
            comparisons += 1
            if text[i+j] != pattern[j]:
                break
            j += 1
        matched = (j == m)
        if matched:
            occurrences.append(i)
            shift_log.append((i, local_comp, matched))

    return occurrences, comparisons, shift_log

text = "TTATAGATCTCGTATTCTTTATAGATCTCCTATTCTT"
pattern = "TATCTT"
n, m = len(text), len(pattern)

occurrences, total_comparisons, shift_log = brute_force_full_analysis(text, pattern)

print("Text   :", text)
print("Pattern:", pattern)
print("Text length (n):", n, " Pattern length (m):", m)
print()

print("---- Shifting Process (shift : comparisons made : result) ----")
for shift, comp, matched in shift_log:
    result = "MATCH" if matched else "mismatch"
    print(f"Shift {shift:2d} -> aligned '{text[shift:shift+m]}' vs '{pattern}' : {comp} comparison(s) : {result}")

print()
print("Total shifts attempted:", len(shift_log))
print("Pattern occurrence position(s):", occurrences if occurrences else "None")
print("Total comparisons performed:", total_comparisons)

best_case = m
worst_case = m * (n - m + 1)
print()
print("---- Complexity Analysis ----")
print(f"Best-case time complexity : O(m)   -> here m={best_case} comparisons if first char always mismatches quickly, or immediate full match at first alignment")
```

```

print(f"Worst-case time complexity:  $O(n*m)$  -> here max possible = {worst_case}
comparisons")
print(f"Actual comparisons in this run: {total_comparisons}")
print("Space complexity:  $O(1)$  additional space (excluding input storage for text and
pattern)")

```

Output

```

Text   : TTATAGATCTCGTATTCTTTATAGATCTCCTATTCTT
Pattern: TATCTT
Text length (n): 37  Pattern length (m): 6

---- Shifting Process (shift : comparisons made : result) ----
Shift 0 -> aligned 'TTATAG' vs 'TATCTT' : 2 comparison(s) : mismatch
Shift 1 -> aligned 'TATAGA' vs 'TATCTT' : 4 comparison(s) : mismatch
Shift 2 -> aligned 'ATAGAT' vs 'TATCTT' : 1 comparison(s) : mismatch
Shift 3 -> aligned 'TAGATC' vs 'TATCTT' : 3 comparison(s) : mismatch
Shift 4 -> aligned 'AGATCT' vs 'TATCTT' : 1 comparison(s) : mismatch
Shift 5 -> aligned 'GATCTC' vs 'TATCTT' : 1 comparison(s) : mismatch
Shift 6 -> aligned 'ATCTCG' vs 'TATCTT' : 1 comparison(s) : mismatch
Shift 7 -> aligned 'TCTCGT' vs 'TATCTT' : 2 comparison(s) : mismatch
Shift 8 -> aligned 'CTCGTA' vs 'TATCTT' : 1 comparison(s) : mismatch
Shift 9 -> aligned 'TCGTAT' vs 'TATCTT' : 2 comparison(s) : mismatch
Shift 10 -> aligned 'CGTATT' vs 'TATCTT' : 1 comparison(s) : mismatch
Shift 11 -> aligned 'GTATTC' vs 'TATCTT' : 1 comparison(s) : mismatch
Shift 12 -> aligned 'TATTCT' vs 'TATCTT' : 4 comparison(s) : mismatch
Shift 13 -> aligned 'ATTCTT' vs 'TATCTT' : 1 comparison(s) : mismatch
Shift 14 -> aligned 'TTCTTT' vs 'TATCTT' : 2 comparison(s) : mismatch
Shift 15 -> aligned 'TCTTTA' vs 'TATCTT' : 2 comparison(s) : mismatch
Shift 16 -> aligned 'CTTTAT' vs 'TATCTT' : 1 comparison(s) : mismatch
Shift 17 -> aligned 'TTTATA' vs 'TATCTT' : 2 comparison(s) : mismatch
Shift 18 -> aligned 'TTATAG' vs 'TATCTT' : 2 comparison(s) : mismatch
Shift 19 -> aligned 'TATAGA' vs 'TATCTT' : 4 comparison(s) : mismatch
Shift 20 -> aligned 'ATAGAT' vs 'TATCTT' : 1 comparison(s) : mismatch
Shift 21 -> aligned 'TAGATC' vs 'TATCTT' : 3 comparison(s) : mismatch
Shift 22 -> aligned 'AGATCT' vs 'TATCTT' : 1 comparison(s) : mismatch
Shift 23 -> aligned 'GATCTC' vs 'TATCTT' : 1 comparison(s) : mismatch
Shift 24 -> aligned 'ATCTCC' vs 'TATCTT' : 1 comparison(s) : mismatch
Shift 25 -> aligned 'TCTCCT' vs 'TATCTT' : 2 comparison(s) : mismatch
Shift 26 -> aligned 'CTCCTA' vs 'TATCTT' : 1 comparison(s) : mismatch
Shift 27 -> aligned 'TCCTAT' vs 'TATCTT' : 2 comparison(s) : mismatch
Shift 28 -> aligned 'CCTATT' vs 'TATCTT' : 1 comparison(s) : mismatch
Shift 29 -> aligned 'CTATTC' vs 'TATCTT' : 1 comparison(s) : mismatch
Shift 30 -> aligned 'TATTCT' vs 'TATCTT' : 4 comparison(s) : mismatch
Shift 31 -> aligned 'ATTCTT' vs 'TATCTT' : 1 comparison(s) : mismatch

Total shifts attempted: 32
Pattern occurrence position(s): None
Total comparisons performed: 57

---- Complexity Analysis ----
Best-case time complexity :  $O(m)$  -> here m=6 comparisons if first char always mismatches
quickly, or immediate full match at first alignment
Worst-case time complexity:  $O(n*m)$  -> here max possible = 192 comparisons
Actual comparisons in this run: 57
Space complexity:  $O(1)$  additional space (excluding input storage for text and pattern)

```

Notes / Analysis

Best-Case Complexity: $O(m)$ — happens when the first character comparison mismatches at nearly every shift, requiring only 1 comparison per shift.

Worst-Case Complexity: $O(n \cdot m)$ — happens when a large prefix of the pattern matches at many shifts before mismatching. For $n=37$, $m=6$, the theoretical maximum is $6 \times 32 = 192$ comparisons.

Actual Result: The program performed 57 comparisons across 32 shifts and did not find the pattern 'TATCTT' anywhere in the given text, since no alignment produces a full character-by-character match.

Space Complexity: $O(1)$ — only a fixed number of index and counter variables are used, regardless of input size; no extra arrays or data structures are allocated.